SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

COURSE NAME: Theory of Computation **COURSE CODE: CSA13**

COURSE OUTCOME COVERED

CO1: *Apply mathematical foundations such as formal proofs, induction, and basic concepts of automata theory to solve computational problems. (BL:3)*

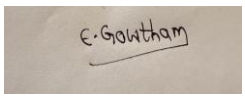
Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks:50**
Duration : 1 Hour

Simulation

Code of Conduct:

I Enamala Gowtham /192311190 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

Criteria	Weightage	Q1 (10)	Q2 (10)	Q3 (10)	Q4 (10)	Q5 (10)	Total Marks (50)
Conceptual Understanding	30	3	3	3	3	3	15
Accuracy of Answer	25	2.5	2.5	2.5	2.5	2.5	12.5
Application of Knowledge	20	2	2	2	2	2	10
Completeness of Response	15	1.5	1.5	1.5	1.5	1.5	7.5
Clarity & Presentation	10	1	1	1	1	1	5
Total per Question	10 Marks	/10	/10	/10	/10	/10	/ 50

1 Design a **Non-Deterministic Finite Automaton (NBA)** for an **Automatic Door System** based on sensor inputs.An automatic door system can be modeled using an NBA to represent different states and transitions. The system consists of four states: *Closed, Opening, Open,* and *Closing*. The input symbols are P (Person detected) and N (No person detected). Initially, the door is in the *Closed* state. When a person is detected (P), the system may non-deterministically transition from *Closed* to *Opening*. Once opened, the system moves to the *Open* state. If no person is detected (N), the door transitions to *Closing* and eventually returns to the *Closed* state. Non-determinism occurs when the system may remain in the *Open* state even if a person is detected or may start closing if no detection occurs.

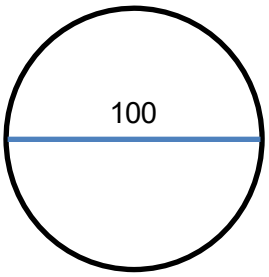
2 Design a finite automaton to model a traffic light control system at a road intersection. The automaton should represent different states corresponding to the traffic lights, such as Red, Green, and Yellow. Define the transitions between these states based on a timer or control signals, ensuring that the sequence follows the standard order (Red → Green → Yellow → Red). Include considerations for timing constraints, such as how long each light remains active, and describe how the automaton handles continuous cycling of signals. Additionally, explain how this model can be extended to handle pedestrian crossing signals or emergency vehicle overrides, ensuring safe and efficient traffic flow.

3 Design DFA using simulator to accept the string the end with ab over set {a,b)
W= aaabab

4 Design a Deterministic Finite Automaton (DFA) using a simulator to accept only the input strings “c”, “bc”, and “bcaaa” over the alphabet {a, b, c}. The automaton should begin from an initial state and transition through intermediate states based on the sequence of input symbols, ensuring that each valid string follows a unique path. For example, the string “c” should be accepted directly from the start state, while “bc” should pass through states representing “b” and then “c”. Similarly, “bcaaa” should be processed through a sequence of states that track each additional “a” after “bc”. Any deviation from these exact patterns should lead to a dead state, ensuring that no other strings are accepted. Clearly define the states, transition functions, start state, and accepting states, and implement the DFA in a simulator such as Autosim to verify its correctness.

5 Design a Deterministic Finite Automaton (DFA) using a simulator to accept all strings over the alphabet {a, b} that begin with either “a” or “b”. The automaton should start in an initial state and immediately transition to an accepting state upon reading the first input symbol, since any valid string must start with either “a” or “b”. After the first symbol is processed, the DFA should remain in an accepting state regardless of the subsequent sequence of “a” and “b”, thereby allowing any combination of characters to follow. Additionally, include a dead state to handle invalid or empty inputs if required by the simulator. Clearly define the states, transition rules, start state, and accepting states, and verify the design using a simulator such as Autosim to ensure that all valid strings are accepted and invalid ones are rejected.

RUBRICS FOR CALCULATION



C01 ASSESSMENT - 2

Name : E. Gawtham

Reg. No : 192311190

Course : Theory of Computation

Code : CSA 1301

NBA for Automatic Door System

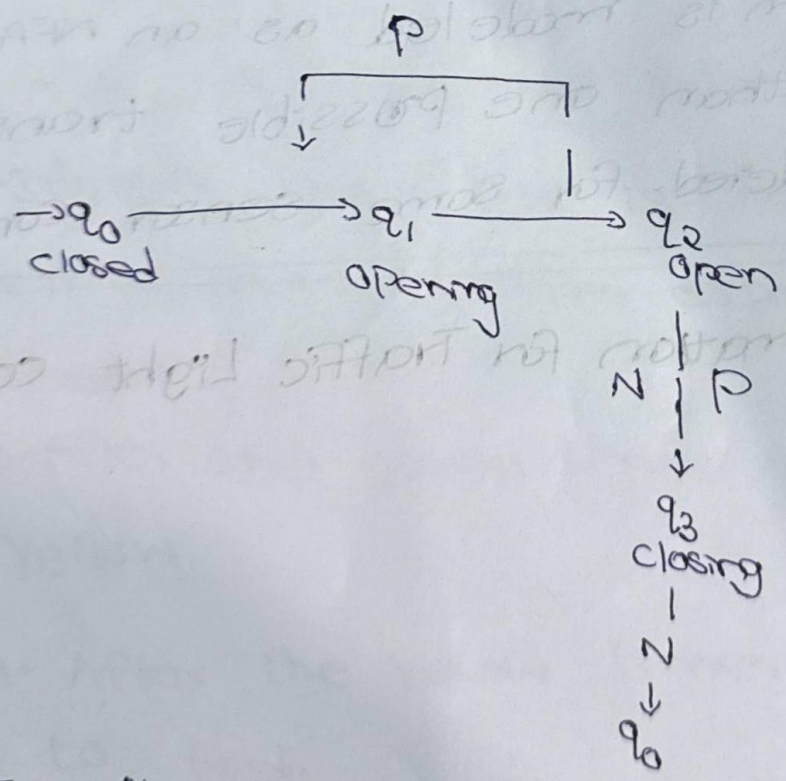
States:

- q_0 = closed - Initial state
- q_1 = Opening
- q_2 = Open
- q_3 = Closing

Input Symbols

- P = Person detected
- N = No person detected

Transition Diagram



Transition Table

Current State	P	N
q_0 (closed)	q_1	q_0

q_1 (opening)	q_2	q_2
q_2 (open)	q_2	q_3
q_3 (closing)	q_1	q_0

Explanation:

Initially, the door is closed (q_0). When q_1 is detected, the door moves to opening (q_1), then reaches open (q_2). If a person is still detected while the door is open, it remains open. When no person is detected, it to moves to closing (q_3) and finally returns to closed (q_0).

The system is modeled as an NFA/NBA because more than one possible transition can be considered for some sensor conditions.

Q2: Finite Automaton for Traffic Light control

states

q_0 = Red

q_1 = Green

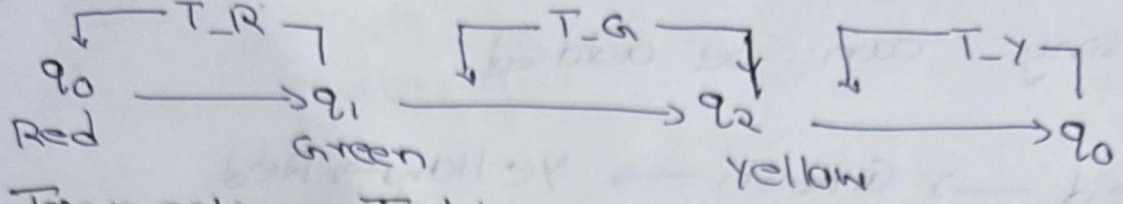
q_2 = yellow

let the timer/ control signals be:

- T_R = Red timer completed
- T_G = Green timer completed

· T_Y = Yellow timer Completed

Transition Diagram



Transition Table

Current State	Input	Next state
Red (q0)	T_R	Green (q1)
Green (q1)	T_G	Yellow (q2)
Yellow (q2)	T_Y	Red (q0)

Working

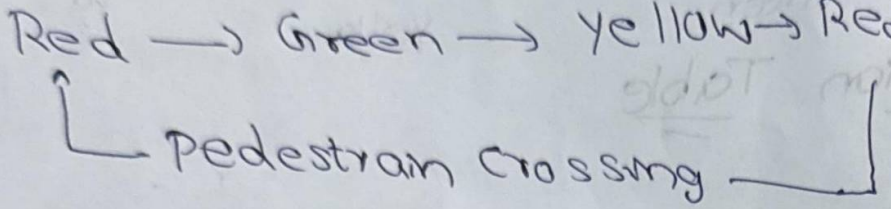
The traffic light follows the standard sequ-ence:

Red → Green → Yellow → Red

1. Initially, the light is Red.
2. After the red timer expires, it changes to Green.
3. After the green timer expires, it changes to Yellow
4. After the yellow timer expires, it returns to Red.
5. The process continuously repeats.

Pedestrian Extension

An additional state such as q_3 = Pedestrian Crossing can be added



During pedestrian crossing, vehicle traffic can be stopped while the pedestrian signal becomes active.

Emergency Override

An emergency input E can immediately force the automaton to the Red state:

Any state $- \epsilon \rightarrow$ Red

This allows emergency vehicles to pass safely.

Q3. DFA to Accept Strings Ending with ab

Given:

Alphabet: $\Sigma = \{a, b\}$

Input String: $w = aaabab$

Required language: Strings that end with ab

States:

q_0 = Nouseful suffix

q_1 = last symbol is a

q_2 = String ends with ab

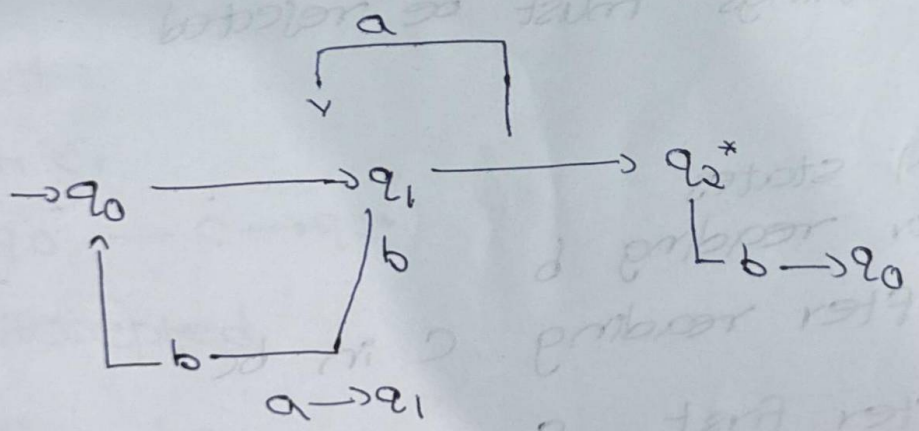
Start state = q_0

final state = q_2

Transition Table

State	a	b
q_0	q_1	q_0
q_1	q_1	q_2
q_2	q_1	q_0

DFA Diagram



Trace for aaabbab

Input	State
Start	q_0
a	q_1
a	q_1
a	q_1
b	q_2

a	q1
b	q2

final state = q2

aaabab is ACCEPTED because it ends in q2

Transition state

→q0

q1

q2

Q4. DFA to ACCEPT only c, bc and bcaaa

Alphabet

$\Sigma = \{a, b, c\}$

Valid strings

c

bc

bcaaa

All other strings must be rejected

States

q_0 = Initial states

q_1 = After reading b

q_2 = After reading c in bc

q_3 = After first a

q_4 = After second a

q_5 = After third a

q_6 = After fourth a

q_D = Dead state

Accepting states:

q_2 and q_6

Transition Table

state	a	b	c
→q0	q0	q1	q2
q1	q0	q0	q2
q2	q3	q0	q0
q3	q4	q0	q0
q4	q5	q0	q0
q5	q6	q0	q0
q6	q0	q0	q0
q0	q0	q0	q0

Paths:

for c:

q0 — c → q2*

Accepted

for bc:

q0 — b → q1 — c → q2*

Accepted

for bcaaa:

q0 — b → q1 — c → q2
- a → q3
- a → q4
- a → q5

$-a \rightarrow q_6^*$

Accepted

Any deviation goes to q_D, so it is rej...

Thus, the DFA accepts only c, bc, and bcaaa

Q5. DFA for strings beginning with a or b

Alphabet

$\Sigma = \{a, b\}$

The language is:

$L = \{w \in \{a,b\}^*\}$

That is, every non-empty string over $\{a, b\}$ is accepted because its first character must be either a or b

States:

q_0 = Initial state

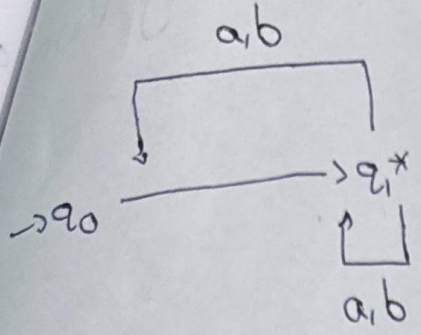
q_1 = Accepting state

q_D = Dead state

Transition Table

State	a	b
$\rightarrow q_0$	q_1	q_1
$*q_1$	q_1	q_1
q_D	q_D	q_D

Diagram



Working:

1. Initially, the DFA is in q0
2. If the first symbol is a or b, it moves to q1.
3. q1 is an accepting state
4. Once q1 is reached, any number of a or b symbols can read while remaining in q1
5. The empty string is rejected because the initial state q0 is not accepting

Examples

String	Result
a	Accept
b	Accept
ab	Accept
ba	Accept
aabb	Accept
bbaa	Accept
abab	Accept

ε . Reject

Therefore, the DFA accepts every non-empty string over {a, b}